arXiv:2303.09331v1 [cs.LG] 16 Mar 2023

Model based Explanations of Concept Drift

Fabian Hinder, Valerie Vaquet,
Johannes Brinkrolf, and Barbara Hammer

{ fhinder, vvaquet, jbrinkro, bhammer}@techfak.uni-bielefeld.de

Bielefeld University - Cognitive Interaction Technology (CITEC)
Inspiration 1, 33619 Bielefeld - Germany

March 17, 2023

Abstract

The notion of concept drift refers to the phenomenon that the distribution generating the observed data changes over time. If drift is present, machine learning models can become inaccurate and need adjustment. While there do exist methods to detect concept drift or to adjust models in the presence of observed drift, the question of *explaining* drift, i.e., describing the potentially complex and high dimensional change of distribution in a human-understandable fashion, has hardly been considered so far. This problem is of importance since it enables an inspection of the most prominent characteristics of how and where drift manifests itself. Hence, it enables human understanding of the change and it increases acceptance of life-long learning models. In this paper, we present a novel technology characterizing concept drift in terms of the characteristic change of spatial features based on various explanation techniques. To do so, we propose a methodology to reduce the explanation of concept drift to an explanation of models that are trained in a suitable way extracting relevant information regarding the drift. This way a large variety of explanation schemes is available. Thus, a suitable method can be selected for the problem of drift explanation at hand. We outline the potential of this approach and demonstrate its usefulness in several examples.

Keywords: Concept Drift · Explainable AI · Explaining Concept Drift

1 Introduction

The world that surrounds us is undergoing continuous changes, which inflict themselves on the increasing amount of available data sources. Those changes occur, for example, in social media or IoT devices, where data is collected over time [1,2]. Such effects – referred to as concept drift – can be induced by several

causes, e.g., seasonal changes, changed demands of individual customers, aging of sensors, etc. When dealing with such non-stationary environments, there are two main problem setups: Autonomously running systems need to robustly solve a given task in the presence of drift, and monitoring systems need to reliably detect anomalous behavior.

The majority of approaches of the first category aim at developing models which adapt in the presence of drift. Usually, the main objective is minimizing the interleaved test-train error. In case autonomous adaption mechanisms fail and human intervention is required, the user is interested in an explanation for potential drops of accuracy [3].

In system monitoring and in drift detection for so-called active methods in non-stationary environments the drift itself is of interest as it might indicate that certain actions have to be taken [4, 5]. For example, in cyber-security settings, drift indicates a potential attack and in the monitoring of critical infrastructure, e.g. electrical grids or water distribution networks, leakages or other failures can cause drift in the observed data [6, 7]. In such cases, more precise information about the drift, ideally some kind of intuitive explanation, can help to minimize the damage caused by a malfunctioning technical system and reduce the wastage of resources by providing more information to the human operator.

In recent years, considerable research has been conducted on explainable AI. Explanation technologies for classical batch machine learning models can be stratified according to different questions, ranging from how an explanation is computed in relation to the given model (e.g. black-box, post-hoc, natively explainable), what is explained (e.g. local or global behavior), to which explanations are used (e.g. feature-based or example-based) [8]. Just as there are several types of machine learning models each with its own strengths, weaknesses, and use cases, there is also a large variety of explanation schemes [9–18], with each putting focus on a different objective and providing different information for the problem at hand.

Research on analyzing and explaining drift is still limited. Methods for an inspection of the most significant aspects. They can explain the drift, currently mostly focusing on comparably narrow aspects, which are usually not sufficient if a precise description of the drift is required, as is typically the case for high dimensional data. They address the questions when the drift occurs (drift detection) and analyze its strength (drift quantification) [19]. First technologies aim for the identification of particularly relevant features, i.e. those where drift occurs [20, 21]. These approaches are accompanied by methods that attempt identification of inconsistencies caused by drift, i.e. identification of those parts of the model which need to be exchanged, or drift localization, i.e., the parts of the data distribution that are undergoing drift [19]. Although they can theoretically take more complex forms of drift into account, they usually do not provide a condensed explanation. However, such are necessary for human users in general, for pure domain experts as an accessible explanation and even more so for the general audience.

The purpose of our contribution is to provide a novel formalization of how to explain observed drift such that informed monitoring of the underlying process

and the characteristics of the change becomes possible, even for high dimensional, non-sematic data, like images. More specifically, we show how to make use of the link between the problems of drift localization [19, 22] and drift segmentation [23] which can be performed by usual probabilistic classification or conditional density estimators, to obtain an efficient algorithmic scheme for drift explanations using various schemes of model explanations. As such characteristics can be complex, they allow for a detailed inspection of the drift while still being understandable for the human user and sufficiently rich to describe the specific problem at hand.

This paper is organized as follows: In Section 2 we recall the formal definition of concept drift (Section 2.1), give a comparative overview of the existing literature for drift explanations (Section 2.2), and provide a high level description of some of the most relevant explanation schemes (Section 2.3) as well as drift localization and segmentation (Section 2.4). In Section 3 we outline the proposed technology in general and then focus on specific problems and instantiations. Then we illustrate the proposed method in Section 4 using several examples, starting from a more quantitative evaluation and proceeding to more and more complex problem setups. Finally, we conclude and point out some further research questions and problems (Section 5).

2 Problem Setup and Related Work

In this work, we propose a methodology for drift explanation relying on model-based approaches which extract spatial properties of drift [22, 23] and several methods for model explanations [9–18]. Before recapping these, we recall the formal framework for concept drift as introduced in the work [24,25] and provide a summary of the related work on explanation methods in the context of concept drift.

2.1 A Statistical Framework for Concept Drift

In the classical batch setup of machine learning one considers a generative process $\mathcal{D}$, i.e., a probability measure, on a data space $\mathcal{X}$. In this context one views the realizations of i.i.d. random variables $X_1, ..., X_n \sim \mathcal{D}$ as samples. However, this setup is not applicable in many real-world applications where the data is arriving consecutively over time. Thus, it is necessary to adapt the problem description: One considers an index set $\mathcal{T}$, representing time, and a collection of (possibly different) distributions $\mathcal{D}_t$ on $\mathcal{X}$, indexed over $\mathcal{T}$ [26]. It is possible to extend this setup to a general statistical interdependence of data and time via a distribution $\mathcal{D}$ on $\mathcal{T} \times \mathcal{X}$ which decomposes into a distribution $\mathbb{P}_T$ on $\mathcal{T}$ and the conditional distributions $\mathcal{D}_t$ on $\mathcal{X}$ such that $X \mid T = t \sim \mathcal{D}_t$ [24, 25].

Drift refers to the fact that $\mathcal{D}_t$ varies for different timepoints, i.e. $\{(t_0, t_1) \in \mathcal{T}^2 : \mathcal{D}_{t_0} \neq \mathcal{D}_{t_1}\}$ has measure larger zero w.r.t $\mathbb{P}_T^2$ [24, 25]. One of the key findings of [24, 25] is a unique characterization of the presence of drift by the

property of statistical dependency of time T and data X if a time-enriched representation of the data $(T, X) \sim \mathcal{D}$ is considered.

2.2 Related Work

While explainability has been a major research interest in recent years [27, 28], explainability methods for drift are still limited. Quite a number of approaches aim for the detection and quantification of drift [19, 29], its localization in space [19], or visualization [20, 20, 30, 31]. Besides, several methods focus on feature-wise representations of drift [20, 29–31]. However, they are limited if dealing with high-dimensional data or non-semantic features. An exception is [21] which takes large-scale feature correlation into account by applying a LIME [9] like procedure to contrastive autoencoders. To our knowledge, this is the only other approach relying on more complex XAI methods for explaining drift. However, this approach is limited to the (semi-)supervised setup and provides only relevant features, which might be less intuitive to humans [32].

To explain drift, more information than that is required to select change points or estimate the rate of change must be extracted. To detect drift, a single drifting feature is sufficient; to explain drift, all are desirable. Drift localization offers such information [22] but has mostly been proposed as a subroutine of drift detection rather than explicit drift explanation technology so far: All methods [33–36] summarized in the overview [19] are restricted to a measurement of the local change of the distribution rather than an explanation by means of XAI techniques, which can track reasonable directions of drift over time. Our proposed method relies on ideas as introduced in the work [22, 23, 37] as a subroutine to extract information regarding the change. We will explain those in more detail in Section 2.4.

2.3 General XAI for Model Explanations

There are several approaches to explainable AI (XAI). Generally, explanation methods can be categorized with respect to the way information is presented: global explanations describe the model as a whole while local or sample-based explanations provide an explanation for a single sample and how the model processed it. In Section 3.2 we will show how to extract particular relevant information about the drift using machine learning models and thereby link the problem of general model explanations and explanation of drift. This way we obtain a very broad range of explanation methods which can be fitted to the specific problem at hand.

In the following, we will recall some of the most relevant approaches and discuss their strengths and weaknesses. In Section 3.4, we will discuss how the methods listed below can be applied and interpreted in the drift-specific setup and also provide an overview regarding the method-specific characteristics (see Table 1). Showcases for all explanation schemes, except for interpretable models which we exclude due to their limited explanatory complexity, are provided in Section 4.

Interpretable Models One of the simplest approaches to explainable AI are white-box or interpretable models [27, 38] which by design allow analysis by the user. Typical models of this category are linear models, decision trees, and prototype-based models. However, whether or not a specific model is actually interpretable depends on the concrete setup. For example on image data, neither linear models nor decision trees are intuitively interpretable due to a large number of features. Complex decision trees with several hundred leaves also lack interpretability. This is due to the fact that the amount of information a human can take in at once is limited. Thus, the complexity of interpretable models – or all global explanations for that sake – has to be rather limited. This in turn limits the complexity of the models and thus the number of applications for this approach.

Discriminative Dimensionality Reduction This group of techniques constitutes another global explanation approach. Standard dimensionality reduction approaches are refined with additional information obtained from a machine learning model. For example in [10,11], the used metric is enriched with information on the decision boundary of the model so that the global decision structure becomes accessible to the user. In contrast to interpretable models, there are no problems with model complexity in this approach. However, it leads to a possible loss of information during the dimensionality reduction process. Thus, for more complex domains or problems, this approach might simplify too much in order to obtain a full picture of the problem at hand.

Global Feature Importance One of the oldest inspection methods is permutation feature importance [12]. The basic idea is to obtain knowledge on the relevance of the single features for the model's internal decision process by permuting the values of a single feature and comparing the model's performance on the modified dataset to the performance on the original dataset. Comparable approaches that are more theoretically grounded are provided by feature relevance theory [13], which is linking conditional independence and graphical models, or Shapley-Values [14], which are resulting from game-theoretic modeling of feature importance from a set of axioms. The drawback of this approach is that it does not provide any information about the single sample as the information is presented in a cumulative fashion only.

In this work, we will mainly focus on permutation feature importance as a very efficient and model-agnostic approach. This method has the benefit that a new variation allows computing those values in an incremental fashion [15] which is of particular relevance in the considered online setup.

Local Feature Importance A counterpart to the global feature importance methods is provided by Saliency Maps [16] or Local Interpretable Model-agnostic Explanations (LIME) [9].

Saliency Maps originate from explainable AI in deep learning, in particular image classification. The idea is to find the most important features of a

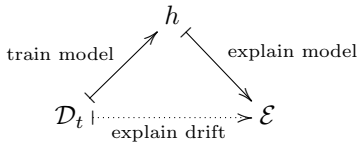


Figure 1: General Scheme for drift explanation: 1. train a model (h) to capture the relevant information of the drift ($\mathcal{D}_t$), 2. extract information from the model (h) via model explanations to obtain an explanation ($\mathcal{E}$) for the drift.

sample by computing the gradient of the classification function with respect to the sample in question. The features associated with the absolute largest partial derivative are then considered as particularly relevant as a change of those would result in a particularly large change of the classification. One obvious drawback of this method is that derivatives are subject to local disturbances of the classification function. This poses a problem as those might be an artifact of the model and do not have any implication on the actual classification.

LIME on the other hand aims to provide an explanation for the classification of a single sample by means of a simplified model, which can be easily interpreted by the user: a model is trained on the complex model's predictions on samples in the proximity of the sample in question. Linear models constitute a common choice for the simplified model. In this case, LIME essentially computes the derivative of the convolution of the model and the sampling distribution. Therefore, LIME suffers – at least to some extent – from the same issues as Saliency Maps. The sampling distribution is a crucial parameter of the method.

Contrasting Explanations and Counterfactuals One way to tackle the problem of local disturbances is provided by counterfactual explanations [17]. While derivatives only provide a hint on how a sample has to be changed in order to obtain a different class counterfactual explanations actually provide a sample of the other class, called a counterfactual. In order to allow the user to grasp the most relevant changes or characteristics of the decision process for the sample at hand, counterfactuals should have additional properties such as being as close as possible to the original sample (closest counterfactual) or lying on the data manifold (plausible counterfactual) [18, 27]. Drawbacks of this method are the computational cost, missing uniqueness of the obtained counterfactuals, and their conceptual closeness to adversarial examples.

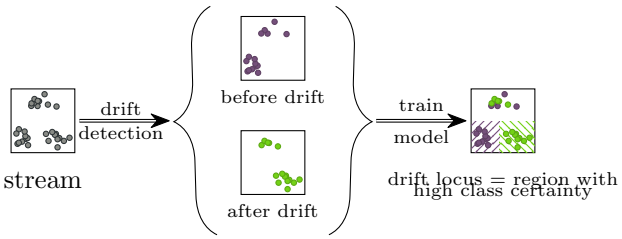


Figure 2: Schematic visualization of drift localization. First, by means of drift detection obtain labeling before/after drift. In a second step, train a model. Regions of high certainty correspond to the drift locus.

2.4 Spatial Properties of Concept Drift: Drift Localization and Drift Segmentation

As stated above the main idea of this contribution is to apply model based explanations to concept drift: First use a model to learn relevant characteristics of the drift and then compute the explanation of the model to understand the drift by proxy (see Figure 1). The tasks which we will consider, aim at an extraction of relevant spatial properties of the drift: Drift Localization and Drift Segmentation. In Section 3.2 we will discuss that such tasks already suffice to obtain and therefore explain all relevant information of the drift. In this section, we recall the general tasks of drift localization and segmentation and elaborate on how they are connected to and can be reduced to a common learning problem.

Drift Localization The problem of identifying whether or not a specific sample is affected by drift or equivalently of finding the regions in dataspace where the drift manifests itself is referred to as drift localization. We illustrated this idea in Figure 2. The task usually assumes a finite collection of timepoints, i.e., $|\mathcal{T}| < \infty$. In the following, we will assume that drift has been detected. This allows us to segment the data stream into the time before and after the drift, corresponding to two timepoints $\mathcal{T} = \{0, 1\}$. Localizing the drift can then be defined as finding a minimal set $L \subset \mathcal{X}$ such that the distributions coincide for the rest: $\mathcal{D}_0(A \setminus L) = \mathcal{D}_1(A \setminus L)$ for all measurable sets A:

Definition 1 ((Minimal) Drift Locus, Drift Localization [22])**.** Let $(\mathcal{D}_t, \mathbb{P}_T)$ be a drift process [24]. This induces the measure $\mathcal{D}_\mathcal{T}(A) := \int \mathcal{D}_t(A) \mathrm{d}\mathbb{P}_T(t)$ as the mean distribution of X over time. A *drift locus* is a measurable set $L \subset \mathcal{X}$ such that $(\mathcal{D}_t(\cdot|L^C), \mathbb{P}_T)$ has no drift and $t \mapsto \mathcal{D}_t(L)$ is $\mathbb{P}_T$-a.s. constant. A drift locus L is *minimal* if it is contained in every other drift locus L' up to a $\mathcal{D}_\mathcal{T}$-null set, i.e., $\mathcal{D}_\mathcal{T}(L \setminus L') = 0$. We refer to the process of finding the minimal drift locus as *drift localization*.

The minimal drift locus L is is uniquely determined by this property. In particular, it can be shown that L is not empty if and only if there is drift [22, Lemma 1].

Algorithm 1 Drift localization.

Input: $S = \{(x_1, t_1), ..., (x_n, t_n)\}$ dated datapoints, θ decision threshold
Output: L drift locus at datapoints
$t_{\text{drift}} \leftarrow \textsc{DriftDetection}(S)$ {Determine change point}
$S' \leftarrow \{(x_i, t'_i) \mid i = 1, \ldots, n\}$ with $t'_i = \mathbf{1}[t_i \geq t_{\text{drift}}]$
$L \leftarrow \textsc{Zeros}(n)$
for all $k = 1, ..., n$ **do**
  $h \leftarrow \textsc{TrainProbabilisticModel}(\{(x_i, t'_i) \in S' \mid i \neq k\})$
  $h_0 \leftarrow \frac{1}{n-1} \sum_{i \neq k} t'_i$
  $L[k] \leftarrow \mathbf{1}[D_{\text{KL}}(h(x_k) \| h_0) \geq \theta]$
end for
return L

Interestingly, drift localization can be realized by training a probabilistic classification model $h : \mathcal{X} \to \Pr(\mathcal{T})$ to predict the time t given a sample x, i.e., whether x was observed before ($t = 0$) or after ($t = 1$) the drift [22, Theorem 2 and Corollary 2]: If the probability coincides with the prior probability up to statistical insufficiencies, i.e., $h(x) \approx \mathbb{P}_T$, then there is no drift and vice versa (see Algorithm 1). Thus, the model that we have to explain is the classifier h.

In the special case of $\mathcal{T} = \{0, 1\}$ we can further refine the drift locus into those parts which are more likely to occur before and those that occur after the drift which we will refer to as *drift regions*.

Notice that by explaining h we can obtain even more fine-grained explanations of the drift than provided by the localization itself as it actually measures the local drift intensity, which is continuous, rather than just the binary drift locus. However, one major drawback of drift localization is that it can only be applied if we consider two timepoints, i.e., "before" and "after" the drift. This requires drift detection, which itself is a non-trivial task.

Drift Segmentation The task of subdividing the dataspace $\mathcal{X}$ into regions or segments of homogeneous drift behavior is referred to as drift segmentation. As discussed in [23] the drifting behavior at the point $x \in \mathcal{X}$ is encoded in the conditional distribution $\mathbb{P}_{T|X=x}$. Thus, formally we want to obtain a map $L : \mathcal{X} \to \mathbb{N}$ such that $L(x) = L(x')$ implies $\mathbb{P}_{T|X=x} = \mathbb{P}_{T|X=x'}$:

Definition 2 (Drift Segmentation [23])**.** For a drift process $(\mathcal{D}_t, \mathbb{P}_T)$ with $(X, T) \sim \mathcal{D}$ a *drift segmentation* of is a measurable map $L : \mathcal{X} \to \mathbb{N}$, which assigns each element of $x \in \mathcal{X}$ an index $L(x)$ corresponding to the segment it belongs to, such that $\mathbb{P}_{T|L(X)} = \mathbb{P}_{T|X}$.

It can be shown that L is a drift segmentation if and only if $T \perp\!\!\!\perp X \mid L(X)$ [23, Lemma 1] which essentially means that if one considers one of the segments $L^{-1}(i),\ i \in \mathbb{N}$ only one does not observe any drift but only a fluctuation of occurrence probability.

In some sense drift segmentation is a continuous extension of drift localization. Indeed, as already pointed out by [23] one can turn any drift segmentation

Algorithm 2 Drift segmentation.

Input: $S = \{(x_1, t_1), ..., (x_n, t_n)\}$ dated datapoints, d degree of preprocessing
Output: L drift segment at datapoints
$S' \leftarrow \{(x_i, (t_i, t_i^2, \ldots, t_i^d))\}$
$h \leftarrow \textsc{TrainSegmentationBasedMultiRegressionModel}(S')$
$l \leftarrow \textsc{ExtractSegmentationFunction}(h)$ $\{h(x) = v \circ l(x)$ with $l : \mathcal{X} \to \mathbb{N}$, $v : \mathbb{N} \to \mathbb{R}^d\}$
$L \leftarrow \textsc{Zeros}(n)$
for all $k = 1, ..., n$ **do**
    $L[k] \leftarrow l(x_k)$
end for
return L

into a drift localization by marking all segments with $\mathbb{P}_{T|L(X)} \neq \mathbb{P}_T$ as drifting. However, besides that, it does not require a finite set of timepoints, and thus drift detection as a prepossessing. Furthermore, drift segmentation is also more fine-grained compared to drift localization. This makes it better suited for continuous time as continuous drift may lead to a situation where the entire dataspace is undergoing drift, although the local drift characteristics might vary a lot.

To obtain a drift segmentation [23] suggested training a special type of decision trees called Kolmogorov-trees, which are trained using the Kolmogorov-Smirnov test, and consider the leaves of those trees as drift segments. Following the ideas of [39] one can also obtain a drift segmentation by training a classical decision tree for the multi-regression problem $X \mapsto (T, T^2, \ldots, T^d)$: By [23, Lemma 1] a map $L : \mathcal{X} \to \mathbb{N}$, which can be given by a decision tree, is a drift segmentation if T and X are independent given $L(X)$. Since L is deterministic, we can measure conditional independence by $\|\mathbb{P}_{T|X} - \mathbb{P}_{T|L(X)}\|_W$. Using [39, Theorem 2] this can be upper bounded by the variance of $(T, T^2, \ldots, T^d)$ given $L(X)$, i.e., the MSE of the obtained decision tree, and a constant term that depends on d and goes to zero for $d \to \infty$.

This once again shows the close connection to drift localization as a decision tree trained for the case $\mathcal{T} = \{0, 1\}, d = 1$ learns the conditional class probability of $T = 1$, i.e., is a probabilistic classifier.

Notice that decision trees are not the only method to obtain a drift segmentation. Indeed, using the ideas from [39, 40] every segmentation-based, multi-regression model gives rise to a drift segmentation (see Algorithm 2).

3 Explaining Concept Drift

When monitoring processes, it is crucial to detect anomalous behavior. Drift detection technologies can automate this step and identify the point in time where the distribution changes [34, 41–46]. However, knowing only the time of



(a) Prototype (b) Counterfactual

Figure 3: Illustration of a representative example for the before drift regions (red foxes will vanish) and associated counterfactual (gray foxes "replace" the red ones).

the drift is usually not sufficient as it often remains unclear how to react to such drift, i.e., to decide whether adapting the model, redoing an analysis, or human intervention is required. While the detected anomaly might be analyzed by an expert, it would be more efficient to obtain explanations in an automated fashion. This would enable a human to initiate an appropriate reaction or, for a layperson, increase an understanding of the necessity to update the model. A drift characterization is particularly demanding for high dimensional data or a lack of clear semantic features. However, even in cases with semantic features capturing the drift, i.e., finding the features affected by the drift, can be challenging in its own right. In particular, this is the case if not the features themselves but only their correlation is affected by the drift.

Although this gives rise to a seemingly large number of problems, many of them can be tackled using a general framework which we are going to present in the following. We will start by outlining the core ideas with an illustrative example. We will then continue by discussing the general algorithm and methodology followed by highlighting some more specific instantiations of this idea.

3.1 Outline of Proposed Method(s): An Example

In this example, we rely on an example-based explanation scheme. We propose to describe the drift characteristics by contrasting suitable representatives of the underlying distributions [17, 27]. Before delving into more details, let us describe the underlying motivation:

Suppose we are considering steams of pictures taken by stationary webcams in a zoo. If the species within a compound are exchanged, say red fox by gray fox, it will cause drift. Assume that we already applied drift detection and obtained the time of the drift, i.e., we are dealing with determined cases "before" and "after". Explaining the drift to a user in an intuitive way can be achieved by presenting two pictures showing the compound before and after the drift, as shown in Figure 3. Inspecting these images, the user can easily spot

the difference and understand that the fox species changed.

In this example, the species can be considered as a feature, that appears or ceases to be. Geometrically speaking, the distribution moves between the regions where the feature is or is not present. Thus, the change can be explained by localizing the drift and representing the localization in an intuitive way. Drift localization can essentially be performed by first training a classifier discriminating between samples collected before and after the drift occurred and then analyzing it (we will discuss the validity of this approach in Section 3.2). The intuition behind this idea is rather simple: Algorithmically we can find characteristics, just as the species in the example above, using machine learning models. If a sample shows a certain feature – for example the species – that did only occur at a certain point in time, that sample cannot be observed at any other date, and can therefore be used to predict the moment of observation.

Finally, a suitable explanation needs to be computed and presented to the user. In this example, we rely on contrastive explanations, since they are considered particularly easy to understand by humans. As the species is the main feature to date the picture, the system would try to produce a counterfactual explanation consisting of the original picture and a counterfactual where the red fox was retouched by a gray fox, allowing the user to grasp the feature "species" as desired.

In order to perform this task in a completely automated fashion we also need to find samples where the drift manifests itself, i.e., show those features in a particularly prominent way. For this, we will rely on weighted, prototype-based clustering methods (see Section 3.3).

By applying this approach to a data stream that simulates our zoo example using ImageNet [47] pictures, we obtain the explanation given in Figure 3 which we discuss in more detail in Section 4.2.3.

3.2 General Algorithm and Methodology

The core idea for our explanation approach is essentially based on an application of the Bayes Theorem. If $\mathcal{D}_t$ is a drift process and X, T are data time pairs drawn from it, i.e., $(X, T) \sim \mathcal{D}$. Then by Bayes Theorem, for time windows $W \subset \mathcal{T}$ and areas in dataspace $A \subset \mathcal{X}$ it holds

$$\mathcal{D}_W(A) = \mathbb{P}[X \in A \mid T \in W] = \frac{\mathbb{P}[X \in A]}{\mathbb{P}[T \in W]} \mathbb{P}[T \in W \mid X \in A].$$

As pointed out by [24], drift is encoded in the dependence of X and T. Thus, the term $\mathbb{P}[X \in A]/\mathbb{P}[T \in W]$ does not contain any information regarding the drift as it does not contain any information regarding the joint distribution of X and T. Hence, the entire information is encoded in $\mathbb{P}[T \in W \mid X \in A]$. As $\mathcal{T}$ is usually a subset of the real or natural number the estimation of $\mathbb{P}[T \mid X]$ is essentially a conditional density estimation or probabilistic classification if $\mathcal{T}$ is finite, respectively. Thus, by training a model $h(t \mid x)$ to estimate $\mathbb{P}[T = t \mid X = x]$ we essentially extract information about the drift.

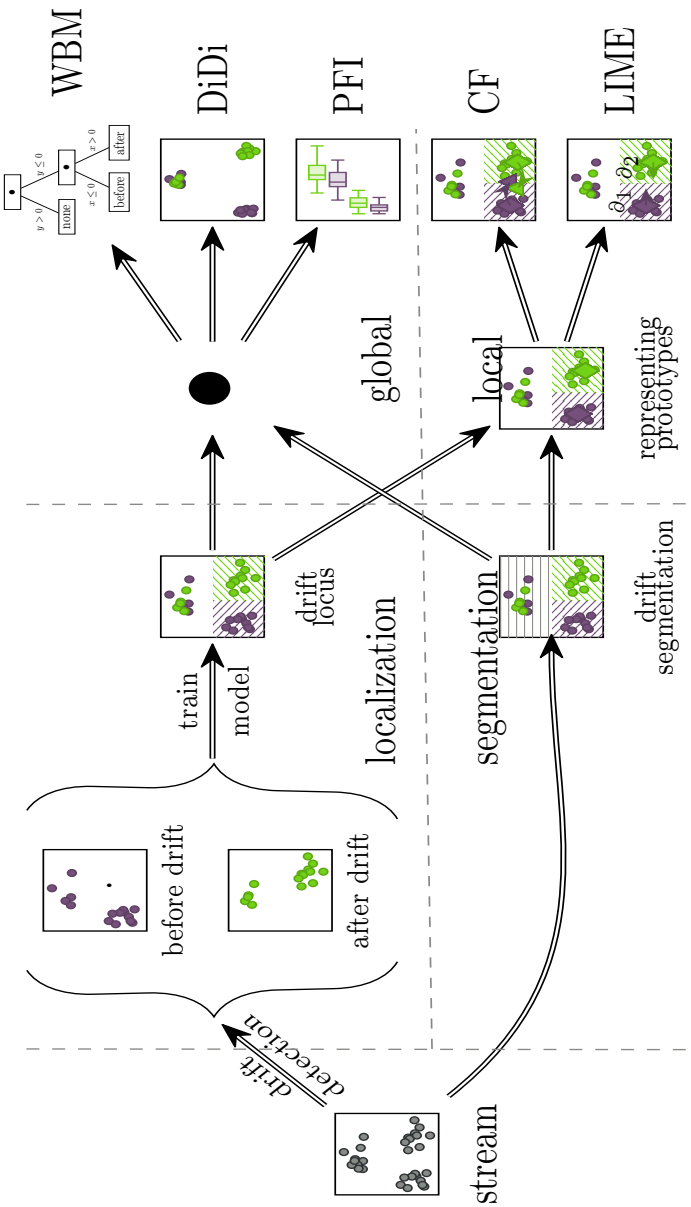


Figure 4: Outline of solutions and processing steps. The stream is first partitioned by means of drift localization or drift segmentation. In a second step, global or local explanation techniques are applied. For local explanations first representing prototypes need to be obtained.

Algorithm 3 Drift Explanation.

1: **Input:** $S = \{(x_1, t_1), ..., (x_n, t_n)\}$ dated datapoints
2: **Output:** $\mathcal{E}$ Explanation
3: $S' \leftarrow$ Preprocess(S)
4: $h \leftarrow$ TrainModel(S')
5: $\mathcal{E} \leftarrow$ ExplainModel($h, \{x_1, \ldots, x_n\}$)
6: **return** $\mathcal{E}$

Indeed, both drift localization and segmentation as approached in [22, 23] and described in Algorithm 1 and 2 perform the task by analyzing such a model for a fix x. This directly shows why those approaches are local in the dataspace and that they are strongly connected to the presence of drift, as in the case of the absence of drift, $h(t \mid x)$ becomes x-invariant.

On an algorithmic level, the problem of drift explanation can be solved by first training a machine learning model to either localize or segment the drift (Line 8 in Algorithm 1 or Line 4 in Algorithm 2) and then analyze it as a proxy to understand the drift as visualized in Figure 4. For localization, one can apply any classification model, while for segmentation any segmentation-based multi-regression model can be used. In a second step, one can choose a suitable explanation method for the concrete problem setup. We already presented a range of candidate techniques in Section 2.3. An inspection and summary on how they can be used in the proposed explanation scheme will be provided in Section 3.4. As many explanation approaches are providing explanations for concrete examples, a selection of suitable samples is crucial. We will elaborate on this in the next section.

A schematic pseudo-code for the explanation routine is given in Algorithm 3 and 4. The function Preprocess is realized by (drift detection and) timepoint transformation (Line 3 in Algorithm 1 and Line 3 in Algorithm 2), the function ExplainModel computes an explanation which is either realized by a global method (Interpretable Model, Global Feature Importance, or Discriminative Dimensionality Reduction) or by Algorithm 4 in case a local explanation (Local Feature Importance, Characteristic Prototypes, Counterfactuals) is applied. In Algorithm 4 characteristic samples (Section 3.3) are computed for each drift segment or region, i.e., all points that belong to the minimal drift locus and are observed before or after the drift. For each prototype obtained this way we compute a local explanation using the respective local explanation method on the model h.

3.3 Characteristic Samples

Many recent powerful explanations methods are local, i.e., they require samples that are to be explained in order to be applicable. The first step of such approaches is thus to determine the samples which are used for the computation of the explanation. In case there is a human in the loop, they can select the most interesting samples. As we aim for a completely autonomous system that

Algorithm 4 Local Model Explanation.

Input: $S = \{x_1, \ldots, x_n\}$ datapoints, h trained model
Output: $\mathcal{E}$ Explanation
$L \leftarrow \textsc{ExtractSegments}(h, S)$ {Drift Segments or Drift Region}
$\mathcal{E} \leftarrow \emptyset$
for all Segments k in L **do**
  $S_k \leftarrow \{x_i \mid L[i] = k\}$
  $C_k \leftarrow \textsc{ClusterPrototypes}(S_k)$
  for all Prototpye $c \in C_k$ **do**
    $\mathcal{E} \leftarrow \mathcal{E} \cup \textsc{ComputeLocalExplanation}(h, c)$
  end for
end for
return $\mathcal{E}$

explains the drift at each time step, it is mandatory to automate this step. In order to be suited for this task such samples $c_1, \ldots, c_n \in \mathcal{X}$ have to fulfill the following requirements:

1. they give rise to a partition of the dataspace, i.e., $P : \mathcal{X} \to \{1, \ldots, n\}$, $P(c_i) = i$
2. they represent all associated datapoints, i.e., if $P(x) = P(c)$ then x is represented by c
3. the drifting behavior of the associated samples is homogeneous, i.e., $\mathbb{P}_{T|X} \approx \mathbb{P}_{T|P(X)}$

We will refer to $c_1, \ldots, c_n$ as *characteristic samples*. As can be seen by the last point the characteristic samples give rise to a drift segmentation. However, as the term "represent" is ill-posed and there are several ways how to construct a partition based on prototypes the term characteristic sample is ill-posed, too.

A common way to construct a partition from prototypes is to associate every point in $\mathcal{X}$ with its closest prototype. In this setup, it is reasonable to measure how well a prototype represents a datapoint using the distance between both. Thus we can find characteristic samples by applying prototype-based clustering algorithms like mean shift, Gaussian mixture models, k-means, affinity propagation, or spectral clustering. In order to assure the time-homogeneity condition is fulfilled we consider only those samples that belong to one drift segment or drift regions at once, i.e., only those samples that are drifting and are observed before the drift. This approach is presented in Algorithm 4.

One drawback of this method is that it is hard to control the number of samples per prototype and assure that the clusterings obtained on different subsets are actually compatible. One way to solve this issue is to draw ideas from the field of discriminative dimensionality reduction where one considers a metric d on $\mathcal{X}$ that takes model-specific information into account [10]. As before, by applying this idea to $h(t \mid x)$ to obtain an enriched metric we also capture the

drifting behavior. To do so we start with a metric on time distributions $d_{\Pr(\mathcal{T})}$ and then consider the difference in prediction which gives rise to a metric on $\mathcal{X}$: $d_L(x, y) = d_{\Pr(\mathcal{T})}(h(\cdot \mid x), h(\cdot \mid y))$. This metric captures the drift exactly at the points x, y to capture the geometry of the entire space one uses the length of the shortest path between x and y [10]. By applying a prototype-based clustering algorithm using this metric we naturally capture both the geometry of $\mathcal{X}$ which is relevant for representing the data and the drifting behavior.

A common approach to compute the distance is to start with a k-neighbour graph, use $d_L(x, y) + \lambda d_{\mathcal{X}}(x, y)$ as weights for the edges, and then apply an all pairs shortest path algorithm. Unfortunately, this approach is comparably computational expensive. A faster approach is to use MomentTrees as model h and consider the random forest kernel [48] of h which captures both the local geometry of $\mathcal{X}$ and the prediction $h(t \mid x)$ at once.

A more direct approach is to make use of prototype-based models h like LVQ or RBF-networks. The characteristic samples are then given by the prototypes. By definition, those give rise to a time-homogeneous partition. However, in some cases, one has to assure that the prototype actually represents the data. This might require an additional representation loss.

3.4 Outline: Role of Different Explanation Methods

In the following, we will list inspection and explanation methods and how they can be interpreted and thus used in the described setup. We will point out some potential use cases and demonstrate some of those in Section 4. We provide an overview in Table 1. Note that all global explanation methods share the advantage that there is no need to select a characteristic sample.

Global Feature Importance allows an inspection of the most relevant features for the drift, i.e., those features where the drift manifests. This can be of particular interest in system monitoring when semantic features are used, e.g., if a feature corresponds to a sensor. More broadly speaking it can be used if the system as a whole is of interest, rather than particular parts of the dataspace. A great advantage of this approach is that it does not need to determine characteristic samples, which makes it comparably lightweight. Together with incremental analysis strategies, it can therefore be applied in an online fashion [15]. We are going to demonstrate this setup is well suited for the detection and localization of sensor faults in Section 4.1.3. Furthermore, this approach is applied in the experiments described in Sections 4.1.1, and 4.1.2.

Discrimenative Dimensionality Reduction allows for an analysis of the geometry of the drift. This can be relevant if models need to be adjusted by hand or if a global understanding of the drift is necessary. It thus can serve as a first inspection approach. Although we do not need to determine characteristic samples, the procedure can be computationally costly, making a real-time implementation difficult. However, as the provided information can be quite dense, this might not be necessary. This scheme is applied in the showcase described in Sections 4.2.2.

Occurrence Profile instead of applying a complete explanation scheme, it

Table 1: List of explanation schemes.

Expl. Scheme	Scope	Explains	Output	Runtime	Used
Interpretable Model	global	decision structure of drift	model description	low	–
Global Feature Importance	global	drifting features	list of features	low	4.1.1, 4.1.2, 4.1.3
Discriminative Dimensionality Reduction	global	geometry of drift	scatter plot	medium	4.2.2
Occurrence Profile	local	local drifting behavior / drift segments	occurrence time diagrams + collection of samples	medium	–
Local Feature Importance	local	drifting features in subspace	list of samples with marked features	medium	4.2.1
Counterfactual Explanations	local	drift induced feature alterations	list of contrasting sample pairs	high	4.2.2, 4.2.3

can also suffice to compute characteristic samples and show them together with the respective occurrence time profile $\mathbb{P}[T \mid X = x]$. This way the process can be monitored in a comparably simple fashion. In particular, this allows to apply more detailed explanations only if requested by the user and is thus comparably computational efficient.

Local Feature Importance can be of interest if the problem is local in the dataspace. This is for example the case if one is not interested in monitoring the system but rather adjusting learning models. Local feature importances can be obtained by applying LIME or Saliency Maps to the model h in order to provide an explanation for the characteristic samples. This way one obtains an explanation in terms of the most relevant features. In contrast to, for example, permutation feature importances, LIME, and Saliency Maps have been shown to be applicable to high-dimensional, non-semantic data like images. One drawback of this approach is that we only know the most important features. We do not observe the effect of the drift on those. Considering our motivating example (Section 3.1): if a fox is marked as relevant for the drift in a stream of images, is it because the fur color changes or because there are no more foxes in the stream after the drift? This scheme is applied in the showcase described in Section 4.2.1.

Counterfactual Explanations are in a sense a perfect match in the case of two timepoints together with drift localization. This is due to the fact that the kind of explanation perfectly matches the type of extracted data: A counterfactual showcases what was changed by the drift in a particular sample. Thus, we can directly observe the effect of the drift. However, it is not necessarily clear

how to generalize this to multiple timepoints or drift segmentation as we have to specify the complementary class the counterfactual is supposed to belong to. Natural choices could be any other segment or a counterfactual per segment, etc. Furthermore, counterfactuals are usually computationally expensive to obtain and the process is usually not flawless. This scheme is applied in the showcases described in Sections 4.2.2, and 4.2.3.

4 Experiments

We evaluate our methods in several experiments. First, we focus on semantic data (Section 4.1). Here, global explanation schemes are suitable. However, when considering non-semantic data of high dimensionality, they are not applicable anymore. Thus, we showcase the suitability of local explanations. As an exemplary data domain, we focus on images (Section 4.2).

4.1 Global Explanations of Semantic Data

In order to evaluate the proposed explanation framework with global explanation methods, we present two experiments in which we control the drift in the data. While we investigate relatively simple drift dynamics by inducing drift by feature perturbations in a first experiment (Section 4.1.1), we consider more complex drift by creating data streams using Bayesian Networks in the second (Section 4.1.2). In both experiments, the main goal of the methodology is to identify the drifting features correctly. Finally, in Section 4.1.3 we show how our explanation framework can be applied in critical infrastructure.

4.1.1 Drift Induced Feature Perturbations

Data In this experiment, we rely on standard benchmark data and artificially induce drift by perturbing a subset of the features. We consider the following synthetic datasets: AGRAWAL [49],MIXED [44], RandomRBF [50], RandomTree [50], and the following real-world benchmark datasets "Electricity market prices" (Elec) [51], "Forest Covertype" (Forest) [52], and "Nebraska Weather" (Weather) [53]. In any case, we consider the joint distribution, i.e., data and label. To remove uncontrolled effects caused by unknown drift in real-world datasets, we apply a permutation scheme [48]. In order to ensure comparability we perform a mean and variance standardization. Then, we draw a sub-stream of size 1000 samples from the stream. Abrupt drift is induced at a randomly chosen point between 1/3 and 2/3 of the stream, using one of the following perturbations applied to a varying number (1-5) of randomly selected features: setting to zero, adding a fixed shift of size 1-5, adding standard Gaussian noise, or feature wise permutation of the values.

Setup In this experiment, we use the following instantiation of methods for our pipeline. We make use of drift segmentation using a Fourier embedding

Table 2: Results of perturbation experiment. Mean over 200 runs and base datasets.

Pert.		PFI	iPFI	FI
Constant 1	DT	0.99±0.06	0.91±0.20	0.78±0.22
	Las	0.50±0.32	0.65±0.42	–
	MLP	0.95±0.17	0.81±0.30	–
	RF	0.99±0.05	0.85±0.25	0.79±0.22
Constant 2	DT	0.74±0.16	0.86±0.19	0.51±0.22
	Las	0.50±0.25	0.65±0.36	–
	MLP	0.90±0.15	0.78±0.25	–
	RF	0.81±0.19	0.85±0.20	0.51±0.26
Constant 3	DT	0.68±0.18	0.82±0.19	0.40±0.23
	Las	0.50±0.22	0.65±0.34	–
	MLP	0.85±0.16	0.77±0.24	–
	RF	0.71±0.23	0.84±0.19	0.40±0.28
Constant 5	DT	0.60±0.19	0.81±0.18	0.27±0.21
	Las	0.50±0.18	0.67±0.34	–
	MLP	0.77±0.18	0.76±0.24	–
	RF	0.69±0.20	0.82±0.19	0.30±0.27
Gaussian Noise 1	DT	0.92±0.21	0.54±0.37	0.90±0.17
	Las	0.50±0.32	0.66±0.41	–
	MLP	0.78±0.26	0.78±0.32	–
	RF	0.99±0.07	0.42±0.34	0.94±0.14
Gaussian Noise 2	DT	0.78±0.20	0.54±0.29	0.83±0.17
	Las	0.50±0.25	0.65±0.36	–
	MLP	0.76±0.19	0.77±0.28	–
	RF	0.87±0.17	0.41±0.27	0.88±0.16
Gaussian Noise 3	DT	0.72±0.20	0.54±0.27	0.81±0.18
	Las	0.50±0.22	0.65±0.35	–
	MLP	0.72±0.19	0.80±0.26	–
	RF	0.79±0.19	0.43±0.25	0.85±0.18
Gaussian Noise 5	DT	0.64±0.18	0.55±0.25	0.77±0.17
	Las	0.50±0.18	0.65±0.34	–
	MLP	0.68±0.17	0.84±0.23	–
	RF	0.69±0.19	0.43±0.22	0.81±0.16
Value Permutation 1	DT	0.66±0.32	0.58±0.34	0.62±0.33
	Las	0.51±0.31	0.67±0.41	–
	MLP	0.59±0.33	0.80±0.30	–
	RF	0.84±0.26	0.43±0.31	0.65±0.34
Value Permutation 2	DT	0.63±0.24	0.58±0.29	0.60±0.25
	Las	0.51±0.25	0.65±0.36	–
	MLP	0.55±0.23	0.78±0.26	–
	RF	0.79±0.22	0.44±0.26	0.63±0.26
Value Permutation 3	DT	0.60±0.21	0.58±0.27	0.57±0.23
	Las	0.49±0.21	0.66±0.34	–
	MLP	0.53±0.22	0.75±0.26	–
	RF	0.74±0.21	0.45±0.25	0.62±0.23
Value Permutation 5	DT	0.57±0.17	0.61±0.25	0.56±0.18
	Las	0.50±0.18	0.66±0.34	–
	MLP	0.51±0.18	0.76±0.24	–
	RF	0.67±0.16	0.49±0.23	0.59±0.18

Pert.		PFI	iPFI	FI
Shift (+1) 1	DT	0.99±0.05	0.98±0.10	0.83±0.21
	Las	0.49±0.31	0.64±0.42	–
	MLP	0.99±0.05	0.86±0.25	–
	RF	1.00±0.00	0.87±0.26	0.87±0.20
Shift (+1) 2	DT	0.89±0.15	0.98±0.09	0.75±0.21
	Las	0.50±0.24	0.64±0.36	–
	MLP	0.87±0.15	0.82±0.22	–
	RF	0.89±0.17	0.84±0.23	0.79±0.21
Shift (+1) 3	DT	0.81±0.17	0.97±0.09	0.70±0.21
	Las	0.51±0.22	0.65±0.35	–
	MLP	0.80±0.18	0.79±0.24	–
	RF	0.82±0.19	0.82±0.21	0.74±0.22
Shift (+1) 5	DT	0.71±0.17	0.94±0.11	0.62±0.19
	Las	0.49±0.19	0.67±0.34	–
	MLP	0.74±0.18	0.80±0.22	–
	RF	0.68±0.19	0.76±0.18	0.69±0.20
Shift (+2) 1	DT	1.00±0.00	0.99±0.09	0.86±0.20
	Las	0.51±0.33	0.64±0.42	–
	MLP	1.00±0.01	0.91±0.21	–
	RF	1.00±0.00	0.94±0.21	0.87±0.19
Shift (+2) 2	DT	0.95±0.12	0.99±0.07	0.77±0.20
	Las	0.50±0.24	0.64±0.36	–
	MLP	0.94±0.10	0.84±0.22	–
	RF	0.95±0.13	0.93±0.18	0.79±0.21
Shift (+2) 3	DT	0.88±0.15	0.98±0.07	0.71±0.20
	Las	0.50±0.22	0.63±0.35	–
	MLP	0.91±0.11	0.81±0.23	–
	RF	0.86±0.18	0.85±0.19	0.76±0.19
Shift (+2) 5	DT	0.74±0.16	0.96±0.08	0.63±0.18
	Las	0.50±0.19	0.66±0.34	–
	MLP	0.87±0.14	0.80±0.23	–
	RF	0.67±0.19	0.72±0.20	0.71±0.19
Shift (+5) 1	DT	1.00±0.00	0.99±0.07	0.90±0.17
	Las	1.00±0.00	0.65±0.42	–
	MLP	1.00±0.00	0.95±0.16	–
	RF	1.00±0.00	0.94±0.21	0.91±0.16
Shift (+5) 2	DT	0.75±0.17	0.99±0.07	0.70±0.20
	Las	0.96±0.11	0.64±0.37	–
	MLP	1.00±0.01	0.88±0.19	–
	RF	0.80±0.20	0.94±0.16	0.79±0.19
Shift (+5) 3	DT	0.67±0.18	0.98±0.09	0.63±0.19
	Las	0.90±0.13	0.66±0.34	–
	MLP	1.00±0.01	0.81±0.25	–
	RF	0.63±0.22	0.89±0.19	0.73±0.20
Shift (+5) 5	DT	0.60±0.16	0.95±0.12	0.59±0.17
	Las	0.82±0.14	0.64±0.35	–
	MLP	1.00±0.01	0.80±0.23	–
	RF	0.50±0.18	0.80±0.20	0.69±0.18

of 5th degree for the time. We consider the following models in a batch and streaming setup, respectively: Decision Tree/Hoeffding Tree (DT), Random Forest/Adaptive Random Forest (RF), Lasso (Las), and Multi Layer Perceptron (MLP; 1-layer, 100 hidden units). To determine the effect of the drift on the feature we make use of the following feature importance measures: permutation feature importance [12] (PFI), incremental permutation feature importance [15] (IPFI; sum over the entire stream), and feature importance (FI; if available for the model).

We evaluate the results using an AUC-ROC score, i.e., rank the features according to the respective importance score and then check how well the ranking aligns with whether a feature is (non-) drifting. We repeated the process 200 times.

Results A summary of the results is shown in Table 2.

As can be seen, except for the value permutation, the number of affected

features has the strongest effect on the performance (the more the harder), followed by the used inspection method and model. The effect of the used dataset is nearly negligible, the effect of the used perturbation depends: it is very similar for setting to zero (Constant) and adding Gaussian Noise, and the Shifts with different intensities which seem to become easier for larger shifts, value permutation appears to be the hardest problem.

Regarding model and inspection method we observe that DT and RF usually perform rather comparably, although RF works better with PFI and FI, whereas DT appears to be more compatible with iPFI. However, this can be caused by both the inspection method and the stream learner. For Las, we observe that for PFI the results are equivalent to random chance, which are consistently outperformed by iPFI. This could be explained by the fact that the problem cannot be learned by a linear model, which is still capable of learning a smaller time window. For DT, RF, and MLP this consideration is inconclusive regarding the mean, but iPFI usually shows a larger variance.

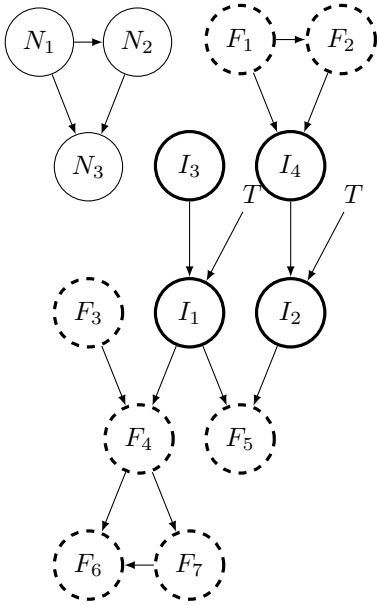
To conclude, the number of affected features has the strongest effect on the result, all other parameters are either negligible or inconclusive. In particular, the incremental approach (iPFI) is not outperformed by the batch methods.

4.1.2 Drifting Bayesian Networks

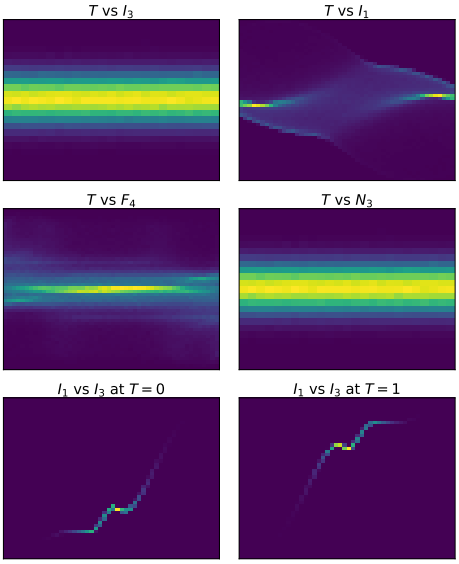
While the last experiment demonstrated that the proposed explanation framework works for simple abrupt drift dynamics, we aim to present its suitability to more complex drift dynamics in this experiment.

Data To generate data streams with more complex drift behavior, we consider randomly constructed Bayesian networks, like the one visualized in Figure 5. Each node takes on a normal distribution, where mean and variance are computed using randomly initialized neural networks. Drift is introduced by making the distribution of some of the nodes time-dependent, i.e., $X_v \mid X_{\mathrm{pa}(v)} \sim p_t(X_v \mid X_{\mathrm{pa}(v)})$. This is realized by making time one of the input features of the network. We illustrated the resulting distribution for some features over time in Figure 5b.

Clearly, all nodes that are directly affected by the time are drifting as well as their children, but also the parent nodes of those as the correlation between the features is affected by the drift. By similar arguments, we inductively obtain all nodes that are in the same connected component as one of the nodes that directly depends on time as drifting. However, it is reasonable to assume that those nodes that are further away from the ones that are directly affected by the drift are "less drifting". To evaluate this effect we run this experiment in two modes. While we generate data based on the entire network in the "complete" setting, we only consider the sub-networks that only consist of nodes that are directly affected by the drift, their parents, and the non-drifting nodes as the "shallow" setup. In Figure 5a the nodes not contained in the shallow setup are marked by a dashed line.



(a) Illustration of random Bayes-Network. Directly time-affected nodes are connected to a T-node (I_1, I_2). Drifting nodes are marked with a thick border line (I_1-I_4, F_1-F_7), non-drifting nodes are marked with a thin border line (N_1-N_3). Nodes with dashed border line are only present in the "complete" setup (F_1-F_7).



(b) Illustration of distribution generated by the network. Upper four pictures show time (T) on x-axis and feature value (I_3, I_1, F_1, N_3) on y-axis. Lower two show feature value of I_3 on x-axis and I_1 on y-axis for different timepoint (left: $T = 0$, right: $T = 1$). As can be seen time has a strong effect on the value of I_1 and the correlation of I_1 and I_3, a small effect on F_4, and no effect on I_3 (on its own) and N_3.

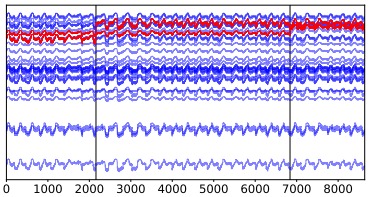
Figure 5: Illustration of Bayes-Network data.

Setup In this experiment, we apply the same methodology as in the last experiment (Section 4.1.1)

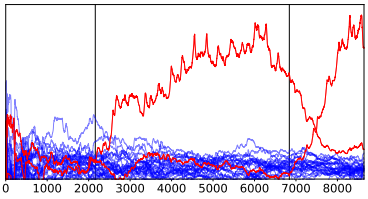
Results The results are shown in Table 3. As can be seen, the shallow network is easier to handle than the complete one. If we consider only the nodes that are directly affected by the drift and their parents as drifting, this difference vanishes. Furthermore, MLP+PFI and Las+iPFI perform best on the complete setup, RF+PFI, DT+iPFI, and Las+iPFI on the shallow setup. MLP and RF are more compatible with PFI than iPFI. For RF and DT, in the complete PFI usually outperforms FI, on the shallow setup this is inconclusive. RF and DT with PFI and FI are usually comparable. To conclude, Las+iPFI performs surprisingly well on many datasets. For the more complex models, MLP and RF are less compatible with iPFI than PFI.

Table 3: Analysis on random Bayesian networks. Mean over 200 runs. Desc. is number of features directly/implicitly/not affected by drift.

Desc.	Model	Complete PFI	Complete iPFI	Complete FI	Shallow PFI	Shallow iPFI	Shallow FI
5/15/5	DT	0.61±0.12	0.20±0.10	0.59±0.12	0.87±0.10	0.74±0.17	0.84±0.09
	Las	0.51±0.15	0.94±0.16	–	0.49±0.19	0.99±0.05	–
	MLP	0.80±0.07	0.11±0.13	–	1.00±0.00	0.55±0.19	–
	RF	0.64±0.12	0.66±0.12	0.61±0.11	0.87±0.10	0.99±0.03	0.88±0.07
7/13/5	DT	0.65±0.11	0.08±0.05	0.55±0.11	0.86±0.09	0.24±0.12	0.84±0.10
	Las	0.51±0.15	0.59±0.37	–	0.49±0.19	0.81±0.24	–
	MLP	0.84±0.07	0.28±0.12	–	0.72±0.05	0.62±0.17	–
	RF	0.66±0.11	0.62±0.14	0.53±0.07	0.90±0.08	0.87±0.09	0.90±0.05
6/11/8	DT	0.58±0.11	0.59±0.15	0.55±0.09	0.70±0.14	0.87±0.11	0.79±0.09
	Las	0.50±0.14	0.99±0.02	–	0.50±0.15	1.00±0.01	–
	MLP	0.69±0.09	0.19±0.06	–	0.63±0.06	0.51±0.11	–
	RF	0.59±0.13	0.49±0.13	0.56±0.06	0.84±0.11	0.59±0.11	0.86±0.04
6/14/5	DT	0.66±0.12	0.55±0.17	0.39±0.10	0.85±0.10	0.93±0.08	0.61±0.10
	Las	0.50±0.15	0.32±0.33	–	0.49±0.17	0.39±0.31	–
	MLP	0.83±0.08	0.15±0.12	–	0.71±0.08	0.46±0.21	–
	RF	0.72±0.12	0.45±0.16	0.33±0.06	0.91±0.08	0.86±0.02	0.61±0.07
6/8/11	DT	0.60±0.10	0.70±0.14	0.64±0.10	0.74±0.10	0.80±0.11	0.81±0.10
	Las	0.50±0.12	0.85±0.24	–	0.51±0.16	0.88±0.20	–
	MLP	0.57±0.06	0.32±0.09	–	0.56±0.05	0.59±0.12	–
	RF	0.62±0.11	0.50±0.12	0.68±0.08	0.78±0.11	0.57±0.11	0.87±0.07
6/7/12	DT	0.64±0.10	0.16±0.12	0.62±0.09	0.74±0.12	0.80±0.13	0.75±0.10
	Las	0.52±0.11	0.67±0.34	–	0.49±0.15	0.79±0.26	–
	MLP	0.72±0.06	0.36±0.16	–	0.76±0.08	0.53±0.13	–
	RF	0.64±0.11	0.62±0.11	0.70±0.07	0.79±0.11	0.74±0.11	0.80±0.08



(a) Raw data



(b) Incremental permutation feature importance

Figure 6: Results of (incremental) feature analysis of water data. Plots show: timepoints of fault (black lines), unaffected features (blue lines), affected features (red lines)

4.1.3 Detection and Localization of Sensor Faults in Water Distribution Networks

Finally, we present a real-world use case for global explanations of concept drift. Critical infrastructures like water distribution networks are usually monitored by several sensors that continuously take measurements of the system. Changes in the reported values indicate a change in the system state that might require manual intervention to assure the integrity of the system or prevent malfunc-

tions. However, as the monitoring system itself can be affected by malfunctions, such as sensor fault, it is important to detect and identify those, i.e., determine the timepoint when the fault happens and identify the broken sensor(s) [54]. This task is non-trivial as the sensor readings are also affected by changing consumer demands which are in turn affected by external factors like the day-night-cycle, workday-and-weekend as well as public holidays, large sports events, or the current weather situation, which also result in complex changes in the sensor readings [55].

Data We generated a dataset of pressure values in the L-Town network using realistic demands [56] for a water distribution network using a commonly used simulation tool from the literature [57]. We add two sensor faults at different timepoints (see Figure 6a).

Setup As each feature corresponds to a single sensor we address the task of sensor fault localization by means of feature importance. We use an adaptive random forest as a model for drift segmentation using a Fourier transform of degree 5 and a period of 500 samples. We analyze the model using incremental permutation feature importances [15].

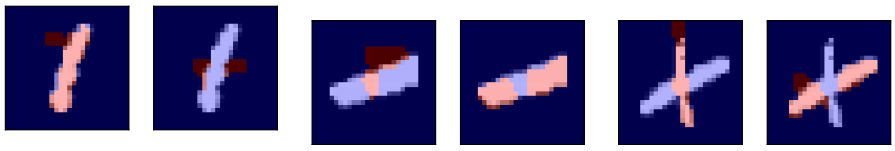
Results The results are presented in Figure 6b. As can be seen, the method correctly identifies the faulty sensors, albeit with some delay. This is consistent with the findings in the first experiment (Section 4.1.1) and we were able to obtain similar results on data generated using different sensors and faults.

4.2 Local Explanations on High-Dimensional Non-Semantic Data

So far we have focused on global explanations and semantic data. Such explanations are comparably simple as it suffices to point out relevant features, either by directly marking them or by showing suitable groupings to point out relevant correlations. However, many real-world data sources do not provide a direct, feature-wise interpretation – especially when they are lacking clear semantics and are high-dimensional. In this section, we will consider image data as an exemplary data domain and explore potential ways to extract drift-related information by local explanation schemes. First, we will focus on rather simple MNIST based scenarios (Section 4.2.1, 4.2.2). Afterward, we will show an example application of the proposed explanation methodology on a more complex data stream (Section4.2.3).

4.2.1 Local Feature-Based Explanations for Non-Sematic Data using MNIST

In this experiment, we explore the potential of feature-based explanations for image data.



(a) Sample from before drift.

(b) Sample from after drift.

(c) Sample from before or after drift.

Figure 7: Explanation for some samples from the MNIST-Plus stream. Images show samples overlayed with LIME relevance profile for "before drift" (left) and "after drift" (right). Red indicates high relevance, blue indicates low relevance.

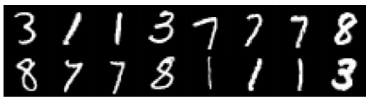
Data We consider a stream with a single abrupt drift, consisting of vertical lines (|), horizontal lines (−), and crosses (+) which carry characteristics of both types. Before the drift, a vertical line (| or +) has to be present, after the drift a horizontal line (− or +) has to be present, each class with 50% rate of occurrence. We obtain the vertical lines as the class "1" from the MNIST [58] dataset, the horizontal lines are obtained by rotating those by 90° clockwise, the pluses are then obtained by randomly choosing a horizontal and vertical line and taking the pixel-wise maximum.

Setup Our goal is to point out regions in an image, i.e., groups of features, that are particularly relevant for the classification. One suitable explanation approach is the LIME-method. We first use an extremely randomized forest as a model for the drift localization and apply LIME to extract the relevant features.

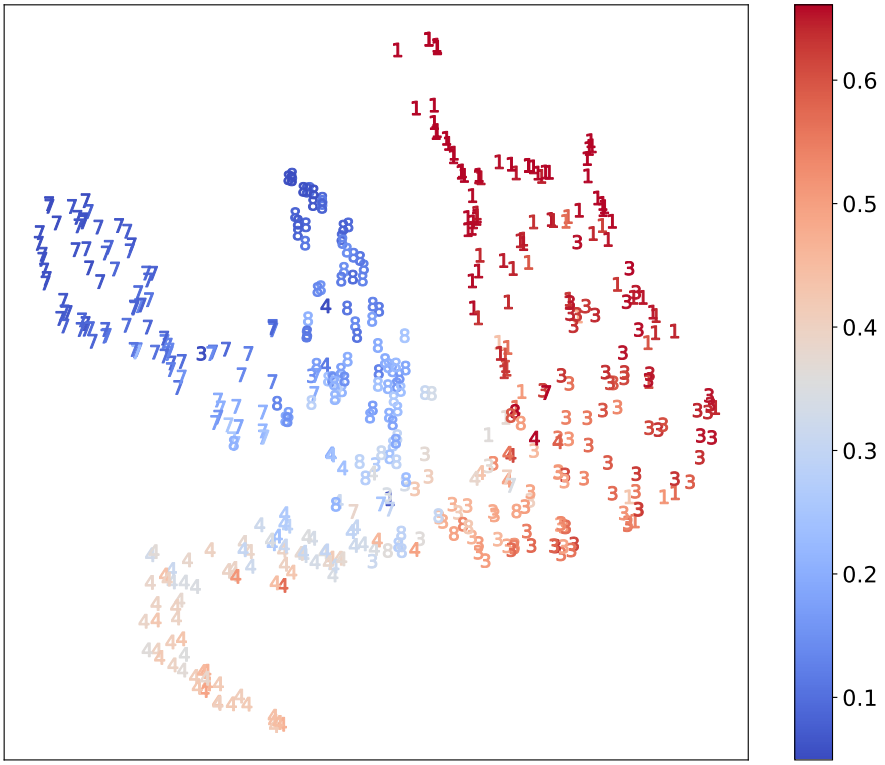
Results Some examples are shown in Figure 7. The left image shows the relevances before the drift, while the right image shows those after the drift. Red indicates high relevance and blue low relevance, respectively. As can be seen, pixels that follow the vertical mid-line are strongly associated with "before drift" whereas pixels to the left and right, in particular in the central region, are associated with "after drift". Thus, the method is capable to capture the main properties of the drift.

4.2.2 Counterfactual Explanations for MNIST Streams

As already pointed out in Section 3.4, feature-based explanations are limited in the sense that they do provide insight about which features are relevant for the drift, but not how the drift affected them. One way to tackle this issue is to use counterfactual explanations which are given by a particular similar sample to the one provided except that they belong to a different class, i.e., show a different drifting behavior in our case. A counterfactual so to speak shows the sample as if observed after the drift, providing more information than just the



(a) Explanation using raw data. The first half of the upper row shows samples before drift, the second half samples after drift respectively. The lower row shows counterfactuals for the samples in the upper row.



(b) Discriminative embedding using t-SNE. Color indicates time-probability ($h(t = 1 \mid x)$), shape MNIST class.

Figure 8: Visualizations of explanations for MNIST Streams

relevant features. In this experiment, we investigate how well this explanation idea translates into practice.

Data To showcase the effect of counterfactual explanations we consider a second stream on a subset of the 28×28-pixel black-white MNIST images. The stream has a single, abrupt drift, the digits "1", "3", and "4" are present before and the digits "7", "8", and "4" after the drift, each with the same rate. Intuitively speaking the drift replaces "1" and "3" by "7" and "8" in the stream.

Setup We use decision trees to extract the drift information and use affinity propagation to select the characteristic samples among the drifting ones, as determined by a drift localization [22]. In order to ensure plausibility, we restricted the set of possible counterfactuals to the training set.

As in this particular problem, a local explanation by dimensionality reduction is suitable as well, we additionally construct a discriminative embedding using a random forest classifier.

Results The counterfactual explanations are presented in Figure 8a. The first half of the upper line presents samples from the data stream before the drift and the second half samples from after the drift. The lower line presents counterfactual explanations of each sample in the upper row, i.e., it shows the user how this sample would have looked like if it had occurred after or before the drift, respectively.

We observe that only the digits "1", "3", "7", "8" are considered to be relevant for the drift. There is also some tendency to associate the digits "1" and "7", and "3" and "8".

The results of the discriminative dimensionality reduction are presented in Figure 8b. As can be seen, the data is separated into three clusters that correlate to the drifting behavior, which overlap mainly at "3", "8" and "4", which can be explained by the optical similarity of "3" and "8" and the fact that "4" is associated with both timepoints. Furthermore, the class probability of the non-drifting samples, i.e., class "4", shows a class probability close to 50% while most samples of the other classes show a strong correlation to the timepoint of observation, as is expected.

As can be seen, the results are very promising though only classical methods are applied. This is not too surprising as MNIST is a rather simple dataset. In the following, we will consider a more complex dataset which can no longer be addressed by classical methods.

4.2.3 Deep Counterfactuals on ImageNet

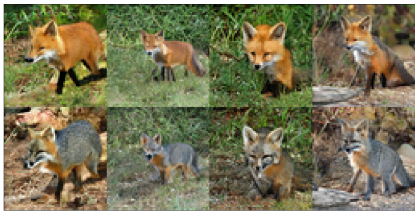
While we were evaluating our methodology on simple image data in the previous two experiments, we now aim to show its application on more complex datasets. For this purpose, we return to our zoo example which was introduced in Section 3.1.

Data We generate a stream considering images of dogs, cats, hamsters, grey, white, and red wolfs, with red foxes only before, and gray foxes only after the drift. All images are taken from the ImageNet [47] dataset which consists of 256×256-pixel color photos. Some exemplary samples of the data stream are shown in Figure 9a. The red line indicates the time of the drift.

Setup We make use of a pretrained VGG16 network for embedding and a BigGAN [59] for the reconstruction. We perform the computations of the counterfactuals in the latent space using decision trees as a model for drift localization



(a) Image stream with drift (marked)



(b) Before → After



(c) After → Before

Figure 9: Illustration of method on ImageNet based data stream. Drift replaces red foxes by gray foxes (a). Explanation (b/c) shows original (Top) and counterfactual (Bottom).

and apply k-means to the drifting samples to obtain the characteristic samples. The counterfactuals are computed using CEML [60].

Results The obtained results are shown in Figures 9b, 9c. Again, the upper rows present the observed samples while the lower rows show the counterfactual explanations of the corresponding samples in the upper row. As can be seen, our method correctly identifies red and gray foxes as the drift-inducing feature, which is then exchanged in the production of the counterfactuals. Notice that the main feature changed by this procedure is the fur color whereas, for example, the posture of the fox is nearly unchanged. However, we made use of a version of BigGAN that respects the image class and thus captured the drift by means of counterfactuals particularly well. It is thus questionable whether or not this approach also works comparably well if we do not consider drift that aligns well with what the deep model is designed to process.

5 Discussion and Future Work

In this work, we considered the problem of explaining concept drift by means of model-based explanations that provide insight into the characteristics of the drift. Our approach is model and explanation independent and we demonstrated its behavior in several examples. The empirical results demonstrate that this proposal constitutes a promising approach as regards drift explanation in an intuitive fashion. The technology is not limited to a finite amount of timepoints nor does it require drift detection in order to be applicable. Although the methodology can be broadly justified theoretically, understanding feature-wise drift behavior is still an unsolved problem which requires a deeper theoretical

understanding. Considering the large variety of graphical models describing real-world data, the proposed data generation in Section 4.1.2 where we used a Bayesian network to create the dataset, could give rise to further theoretical and methodological research in the area of structured analysis of drifting features.

Moreover, applying the technology to complex data such as images requires appropriate preprocessing, e.g., by a deep neural network, which may render some of the explanatory methods useless. Future work could try to deal with this problem by taking ideas from transfer learning into account. Since deep convolutional networks tend to learn a universal representation of the data they might be useful as a universal preprocessing for the data at hand. Besides, examining our proposed methodology in additional important real-world tasks, as for example in anomaly detection and explainable online learning models might be a promising research direction.

Acknowledgement

Funding in the frame of the BMBF project TiM, 05M20PBA and the BMWi project KI-Marktplatz, 01MK20007E is gratefully acknowledged.

References

[1] A. Bifet, J. Gama, Iot data stream analytics, Ann. des Télécomm. 75 (9-10). doi:10.1007/s12243-020-00811-1.
URL https://doi.org/10.1007/s12243-020-00811-1

[2] S. Tabassum, F. S. F. Pereira, S. Fernandes, J. Gama, Social network analysis: An overview, Wiley Interdiscip. Rev. Data Min. Knowl. Discov. 8 (5). doi:10.1002/widm.1256.
URL https://doi.org/10.1002/widm.1256

[3] G. Ditzler, M. Roveri, C. Alippi, R. Polikar, Learning in nonstationary environments: A survey, IEEE Comp. Int. Mag. 10 (4). doi:10.1109/MCI.2015.2471196.
URL https://doi.org/10.1109/MCI.2015.2471196

[4] S. Aminikhanghahi, D. J. Cook, A survey of methods for time series change point detection, Knowl. Inf. Syst. 51 (2). doi:10.1007/s10115-016-0987-z.
URL https://doi.org/10.1007/s10115-016-0987-z

[5] I. Goldenberg, G. I. Webb, Survey of distance measures for quantifying concept drift and shift in numeric data, Knowl. Inf. Syst. 60 (2). doi:10.1007/s10115-018-1257-z.
URL https://doi.org/10.1007/s10115-018-1257-z

[6] H. Haes Alhelou, M. E. Hamedani-Golshan, T. C. Njenda, P. Siano, A survey on power system blackout and cascading events: Research motivations and challenges, Energies 12 (4) (2019) 682.

[7] D. G. Eliades, M. M. Polycarpou, A Fault Diagnosis and Security Framework for Water Systems, IEEE Transactions on Control Systems Technology 18 (6) (2010) 1254–1265. doi:10.1109/TCST.2009.2035515.

[8] R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, F. Giannotti, D. Pedreschi, A survey of methods for explaining black box models, ACM Comput. Surv. 51 (5). doi:10.1145/3236009.
URL https://doi.org/10.1145/3236009

[9] M. T. Ribeiro, S. Singh, C. Guestrin, "why should i trust you?": Explaining the predictions of any classifier, Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.

[10] A. Schulz, F. Hinder, B. Hammer, Deepview: Visualizing classification boundaries of deep neural networks as scatter plots using discriminative dimensionality reduction, in: C. Bessiere (Ed.), Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI-20, International Joint Conferences on Artificial Intelligence Organization, 2020, pp. 2305–2311, main track. doi:10.24963/ijcai.2020/319.
URL https://doi.org/10.24963/ijcai.2020/319

[11] J. Venna, J. Peltonen, K. Nybo, H. Aidos, S. Kaski, Information retrieval perspective to nonlinear dimensionality reduction for data visualization., Journal of Machine Learning Research 11 (2).

[12] L. Breiman, Random forests, Machine learning 45 (1) (2001) 5–32.

[13] R. Nilsson, J. M. Pena, J. Björkegren, J. Tegnér, Consistent feature selection for pattern recognition in polynomial time, The Journal of Machine Learning Research 8 (2007) 589–612.

[14] L. S. Shapley, Notes on the N-person Game–I: Characteristic-point Solutions of the Four-person Game, Rand Corporation, 1951.

[15] F. Fumagalli, M. Muschalik, E. Hüllermeier, B. Hammer, Incremental permutation feature importance (ipfi): Towards online explanations on data streams, arXiv preprint arXiv:2209.01939.

[16] K. Simonyan, A. Vedaldi, A. Zisserman, Deep inside convolutional networks: Visualising image classification models and saliency maps, arXiv preprint arXiv:1312.6034.

[17] S. Wachter, B. D. Mittelstadt, C. Russell, Counterfactual explanations without opening the black box: Automated decisions and the GDPR, CoRR abs/1711.00399. arXiv:1711.00399.
URL http://arxiv.org/abs/1711.00399

[18] A. Looveren, J. Klaise, Interpretable counterfactual explanations guided by prototypes, CoRR abs/1907.02584.

[19] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, G. Zhang, Learning under concept drift: A review, IEEE TKDEdoi:10.1109/tkde.2018.2876857.
URL http://dx.doi.org/10.1109/TKDE.2018.2876857

[20] G. I. Webb, L. K. Lee, F. Petitjean, B. Goethals, Understanding concept drift, CoRR abs/1704.00362. arXiv:1704.00362.
URL http://arxiv.org/abs/1704.00362

[21] L. Yang, W. Guo, Q. Hao, A. Ciptadi, A. Ahmadzadeh, X. Xing, G. Wang, CADE: Detecting and explaining concept drift samples for security applications, in: 30th USENIX Security Symposium (USENIX Security 21), USENIX Association, 2021, pp. 2327–2344.
URL https://www.usenix.org/conference/usenixsecurity21/presentation/yang-limin

[22] F. Hinder, V. Vaquet, J. Brinkrolf, A. Artelt, B. Hammer, Localization of concept drift: Identifying the drifting datapoints, in: 2022 International Joint Conference on Neural Networks (IJCNN), 2022, pp. 1–9. doi:10.1109/IJCNN55064.2022.9892374.

[23] F. Hinder, B. Hammer, M. Verleysen, Concept drift segmentation via kolmogorov-trees., in: ESANN, 2021.

[24] F. Hinder, A. Artelt, B. Hammer, Towards non-parametric drift detection via dynamic adapting window independence drift detection (dawidd), in: ICML, 2020.

[25] F. Hinder, A. Artelt, B. Hammer, A probability theoretic approach to drifting data in continuous time domains, arXiv preprint arXiv:1912.01969.

[26] J. a. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, A. Bouchachia, A survey on concept drift adaptation, ACM Comput. Surv. 46 (4). doi:10.1145/2523813.
URL http://doi.acm.org/10.1145/2523813

[27] C. Molnar, Interpretable Machine Learning, 2020, https://christophm.github.io/interpretable-ml-book/.

[28] K. J. Rohlfing, P. Cimiano, I. Scharlau, T. Matzner, H. M. Buhl, H. Buschmeier, E. Esposito, A. Grimminger, B. Hammer, R. Häb-Umbach, I. Horwath, E. Hüllermeier, F. Kern, S. Kopp, K. Thommes, A. N. Ngomo, C. Schulte, H. Wachsmuth, P. Wagner, B. Wrede, Explanation as a social practice: Toward a conceptual framework for the social design of AI systems, IEEE Trans. Cogn. Dev. Syst. 13 (3) (2021) 717–728. doi:10.1109/TCDS.2020.3044366.
URL https://doi.org/10.1109/TCDS.2020.3044366

[29] G. Webb, L. Lee, B. Goethals, F. Petitjean, Analyzing concept drift and shift from sample data, Data Mining and Knowledge Discovery 32. doi:10.1007/s10618-018-0554-1.

[30] X. Wang, W. Chen, J. Xia, Z. Chen, D. Xu, X. Wu, M. Xu, T. Schreck, Conceptexplorer: Visual analysis of concept drifts in multi-source time-series data, 2020 IEEE Conference on Visual Analytics Science and Technology (VAST).

[31] K. B. Pratt, G. Tschapek, Visualizing concept drift, in: Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '03, Association for Computing Machinery, New York, NY, USA, 2003. doi:10.1145/956750.956849.
URL https://doi.org/10.1145/956750.956849

[32] R. M. J. Byrne, Counterfactuals in explainable artificial intelligence (xai): Evidence from human reasoning, in: Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI-19, International Joint Conferences on Artificial Intelligence Organization, 2019, pp. 6276–6282. doi:10.24963/ijcai.2019/876.
URL https://doi.org/10.24963/ijcai.2019/876

[33] A. Liu, Y. Song, G. Zhang, J. Lu, Regional concept drift detection and density synchronized drift adaptation, in: IJCAI, 2017. doi:10.24963/ijcai.2017/317.
URL https://doi.org/10.24963/ijcai.2017/317

[34] L. Bu, C. Alippi, D. Zhao, A pdf-free change detection test based on density difference estimation, IEEE Trans. Neural Netw. Learn. Syst. 29 (2). doi:10.1109/TNNLS.2016.2619909.

[35] T. Dasu, S. Krishnan, S. Venkatasubramanian, K. Yi, An information-theoretic approach to detecting changes in multidimensional data streams, Interfaces.

[36] N. Lu, J. Lu, G. Zhang, R. L. de Mántaras, A concept drift-tolerant case-base editing technique, Artif. Intell. 230 (2016) 108–133. doi:10.1016/j.artint.2015.09.009.
URL https://doi.org/10.1016/j.artint.2015.09.009

[37] F. Hinder, A. Artelt, V. Vaquet, B. Hammer, Contrasting explanation of concept drift, in: 30th European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning, ESANN, 2022.

[38] M. Du, N. Liu, X. Hu, Techniques for interpretable machine learning, Communications of the ACM 63 (1) (2019) 68–77.

[39] F. Hinder, V. Vaquet, J. Brinkrolf, B. Hammer, Fast non-parametric conditional density estimation using moment trees, in: 2021 IEEE Symposium Series on Computational Intelligence (SSCI), IEEE, 2021, pp. 1–7.

[40] R. Izbicki, A. B. Lee, Converting high-dimensional regression to high-dimensional conditional density estimation, Electronic Journal of Statistics 11 (2) (2017) 2800–2831.

[41] M. Baena-García, J. Campo-Ávila, R. Fidalgo-Merino, A. Bifet, R. Gavald, R. Morales-Bueno, Early drift detection method.

[42] A. Bifet, R. Gavalda, Learning from time-changing data with adaptive windowing, in: SIAM SDM, 2007.

[43] G. Ditzler, R. Polikar, Hellinger distance based drift detection for nonstationary environments, in: IEEE CIDUE, 2011.

[44] J. Gama, P. Medas, G. Castillo, P. Rodrigues, Learning with drift detection, in: Brazilian symposium on artificial intelligence, Springer, 2004.

[45] E. S. PAGE, Continuous inspection schemes, Biometrika 41 (1-2). arXiv:http://oup.prod.sis.lan/biomet/article-pdf/41/1-2/100/1243987/41-1-2-100.pdf, doi:10.1093/biomet/41.1-2.100.
URL https://doi.org/10.1093/biomet/41.1-2.100

[46] A. Wald, Sequential tests of statistical hypotheses, The Annals of Mathematical Statistics 16 (2).
URL http://www.jstor.org/stable/2235829

[47] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, L. Fei-Fei, Imagenet: A large-scale hierarchical image database, in: 2009 IEEE conference on computer vision and pattern recognition, Ieee, 2009, pp. 248–255.

[48] F. Hinder, V. Vaquet, B. Hammer, Suitability of different metric choices for concept drift detection, in: International Symposium on Intelligent Data Analysis, Springer, 2022, pp. 157–170.

[49] R. Agrawal, T. Imielinski, A. N. Swami, Database mining: A performance perspective, IEEE Trans. Knowl. Data Eng. 5 (1993) 914–925.

[50] J. Montiel, J. Read, A. Bifet, T. Abdessalem, Scikit-multiflow: A multi-output streaming framework, Journal of Machine Learning Research 19 (72) (2018) 1–5.
URL http://jmlr.org/papers/v19/18-251.html

[51] M. Harries, N. Wales, Splice-2 comparative evaluation: Electricity pricing (1999).

[52] J. A. Blackard, D. J. Dean, C. W. Anderson, Covertype data set (1998).
URL https://archive.ics.uci.edu/ml/datasets/Covertype

[53] R. Elwell, R. Polikar, Incremental learning of concept drift in nonstationary environments, IEEE Transactions on Neural Networks 22 (10). doi:10.1109/TNN.2011.2160459.

[54] V. Vaquet, A. Artelt, J. Brinkrolf, B. Hammer, Taking care of our drinking water: Dealing with sensor faults in water distribution networks, in: E. Pimenidis, P. P. Angelov, C. Jayne, A. Papaleonidas, M. Aydin (Eds.), Artificial Neural Networks and Machine Learning - ICANN 2022 - 31st International Conference on Artificial Neural Networks, Bristol, UK, September 6-9, 2022, Proceedings, Part II, Vol. 13530 of Lecture Notes in Computer Science, Springer, 2022, pp. 682–693. doi:10.1007/978-3-031-15931-2_56.
URL https://doi.org/10.1007/978-3-031-15931-2_56

[55] E. Vonk, D. G. Cirkel, M. Blokker, Estimating Peak Daily Water Demand under Different Climate Change and Vacation Scenarios, Water 11 (9) (2019) 1874. doi:10.3390/w11091874.
URL https://www.mdpi.com/2073-4441/11/9/1874

[56] S. G. Vrachimis, D. G. Eliades, R. Taormina, Z. Kapelan, A. Ostfeld, S. Liu, M. Kyriakou, P. Pavlou, M. Qiu, M. M. Polycarpou, Battle of the leakage detection and isolation methods, Journal of Water Resources Planning and Management 148 (12) (2022) 04022068. doi:10.1061/(ASCE)WR.1943-5452.0001601.

[57] K. A. Klise, R. Murray, T. Haxton, An overview of the water network tool for resilience (wntr).

[58] Y. LeCun, C. Cortes, MNIST handwritten digit database [cited 2016-01-14 14:24:11].
URL http://yann.lecun.com/exdb/mnist/

[59] A. Brock, J. Donahue, K. Simonyan, Large scale GAN training for high fidelity natural image synthesis, in: International Conference on Learning Representations, 2019.
URL https://openreview.net/forum?id=B1xsqj09Fm

[60] A. Artelt, Ceml-counterfactuals for explaining machine learning models-a python toolbox.